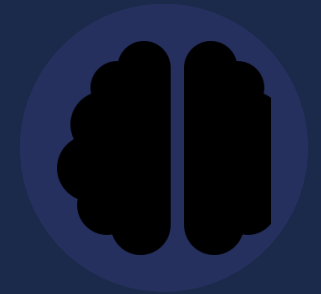


MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition



Session 12

Saga Orchestration

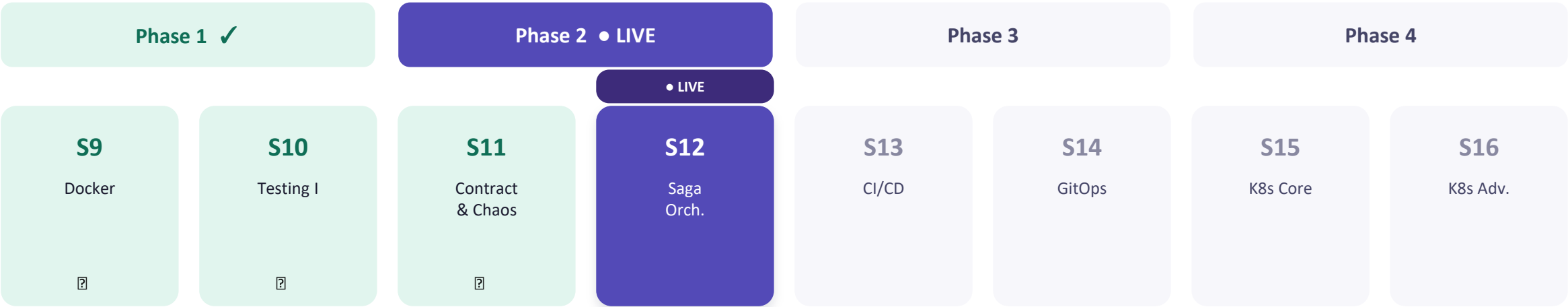
Spring Boot State Machine · SagaState · Commands vs Events



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

Phase 2 — Quality & Deployment



Phase 2 — Quality & Deployment · 8 sessions · S9-S11 complete — today we add explicit Saga control (Orchestration)

Full 29-session roadmap is in the Kickoff deck only.

Why Does Session 12 Exist If Session 7 Solved Distributed Transactions?

Session 7 built:

Choreography Saga — each service reacts to events independently, no central coordinator.

Works well when the flow is simple (≤ 3 services), the team owns all services, and failures are rare.

Session 12 solves:

What happens when the flow grows to 5+ steps, conditions branch mid-saga, and you need ONE place to see the current state of any running transaction?

Session 12 adds: Orchestration Saga — a single `OrderSagaOrchestrator` class owns the full decision tree.

Session 7 code is NOT deleted — both patterns coexist. The Design Choice slide tells you when to use each.

The 5-Service Conditional Flow

1

Order flow grows to 5 steps: Reserve Inventory → Fraud Check → Loyalty Points → Process Payment → Schedule Delivery.

2

Fraud Check has a rule: if fraud risk > 0.7 AND order total > 500 EGP, skip loyalty points and go direct to payment.

3

In Choreography: WHICH service implements this conditional logic? Fraud Check? Loyalty Points? No one owns it clearly.

4

Business logic is now DISTRIBUTED across 5 services. When Order ORD-456 gets stuck, where do you even look?

Three Failure Modes of Choreography at Scale



Invisible State

No single place knows 'Order ORD-123 is at step 3 of 5, waiting for Fraud Check.' Debugging means reconstructing the sequence from 5 services' logs.



Conditional Logic Sprawl

Business rules like 'if fraud > 0.7 then skip loyalty' live inside individual service listeners — scattered, harder to change and test.



Compensation Ambiguity

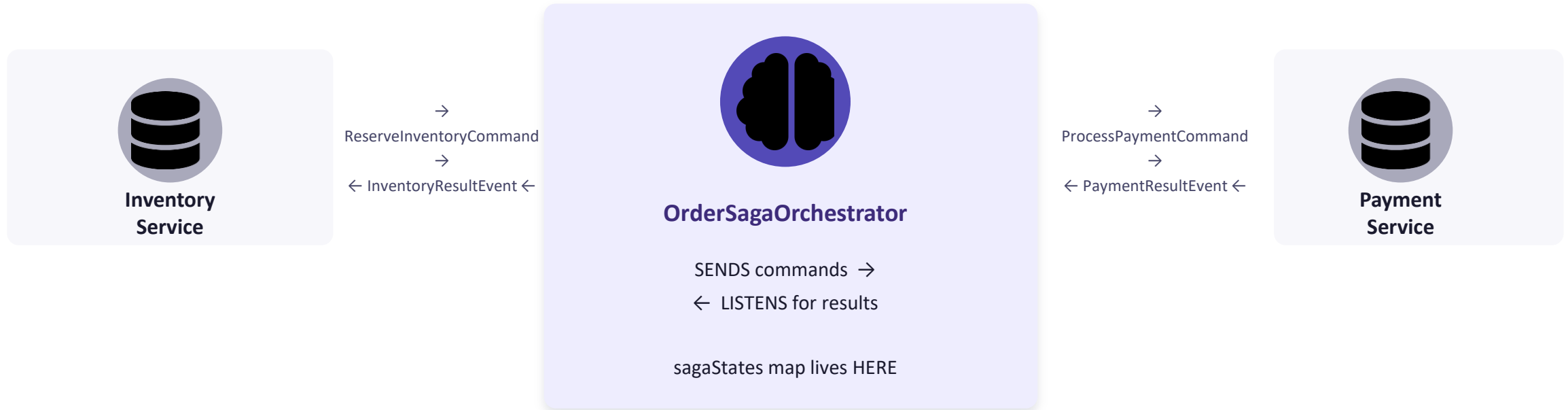
Who decides to release inventory when step 4 fails? If the rule changes, you may need to change 2+ services instead of one.

Choreography (S7) vs Orchestration (S12)

Flow ≤ 3 services, team owns all services?	Choreography
Complex conditional flow (4+ branching steps)?	Orchestration
Need full audit trail of Saga state?	Orchestration
Avoiding single point of failure risk?	Choreography
Team new to event-driven patterns?	Start Choreography
Multiple teams, different deployment schedules?	Choreography
Business rules that change frequently?	Orchestration

Key insight: these are not competing patterns — they solve different problems. Many production systems use BOTH.

The Orchestrator Is the Decision Maker



Inventory Service and Payment Service never talk to each other. Both talk ONLY to the Orchestrator — which decides what happens next.

The Orchestrator Is the Decision Maker

CHOREOGRAPHY (Session 7):

Each service decides its own next action

Order Service —[OrderPlaced]→

Inventory hears → decides → publishes

Payment hears → decides → publishes

Order hears → decides → updates

WHO DECIDES? Each service independently

WHERE IS THE STATE? Scattered everywhere

ORCHESTRATION (Session 12):

Orchestrator decides everything

OrderSagaOrchestrator

SENDS commands (KafkaTemplate)

→ 'Reserve inventory for ORD-123'

→ 'Process payment for ORD-123'

→ 'Release inventory for ORD-123'

LISTENS for results (@KafkaListener)

← 'InventoryReserved' → next step

← 'PaymentFailed' → compensate

← 'InventoryReleased' → cancel order

WHO DECIDES? Orchestrator ALONE

WHERE IS STATE? sagaStates map here

The Two Roles of the Orchestrator



Role 1 — Command Sender

`kafkaTemplate.send()` tells services what to DO.

Does NOT wait for a response on the same thread. Sends and moves on.

Different from OpenFeign (S6): Feign is synchronous. Kafka commands are async.



Role 2 — Result Listener

`@KafkaListener` hears what HAPPENED.

When a result arrives, the Orchestrator updates SagaState and sends the next command.

The Orchestrator decides what happens next — NOT the service that published the result.

“The Orchestrator is the BRAIN. Services are the HANDS. Hands do not choose what to pick up.”

Side-by-Side: Who Decides, Where's the State?

Question	Choreography (S7)	Orchestration (S12)
Who decides next step?	Each service (after receiving event)	Orchestrator exclusively
Where does business logic live?	Distributed across service listeners	Centralized in Orchestrator
How does Inventory know to release stock?	Listens for PaymentFailed event	Orchestrator sends ReleaseStock command
Where is current Saga state?	Implicit — scattered across DBs/logs	Explicit — sagaStates map in Orchestrator
What happens if Orchestrator crashes?	N/A (no coordinator)	In-memory state is lost (see Tech Debt)

SagaState enum — Design the States First

`SagaState.java` — `order-service`

```
public enum SagaState {  
    // — Happy Path States —————  
    STARTED,                // Saga initiated — ReserveInventory sent  
    INVENTORY_RESERVING,    // Waiting for Inventory Service to confirm  
    INVENTORY_RESERVED,    // Inventory confirmed → ProcessPayment sent  
    PAYMENT_PROCESSING,    // Waiting for Payment Service to confirm  
    COMPLETED,            // Payment confirmed → Order is CONFIRMED  
  
    // — Compensation States —————  
    INVENTORY_RESERVE_FAILED, // Inventory said 'no stock' → cancel directly  
    PAYMENT_FAILED,          // Payment failed after inventory was reserved  
    INVENTORY_RELEASING,     // Compensation in progress  
    CANCELLED                // Compensation complete — Order is CANCELLED  
}
```

The State Diagram IS the Design



Compensation Path — triggered when PAYMENT_PROCESSING fails

Ask trainees to draw the state transitions on paper before looking at the code: what states exist? what events cause transitions? Takes 3 minutes and prevents half the bugs.

OrderSagaOrchestrator — Skeleton + startSaga()

OrderSagaOrchestrator.java — order-service

```
package com.microservices.pro.order.saga;

@Service
@Slf4j
public class OrderSagaOrchestrator {

    // DEV ONLY: in-memory state — lost on restart.
    // Production: persist SagaState to DB (see Homework §8).
    private final Map<String, SagaState> sagaStates = new ConcurrentHashMap<>();

    @Autowired
    private KafkaTemplate<String, Object> kafkaTemplate;

    @Autowired
    private OrderRepository orderRepository;
```

DEV ONLY — *ConcurrentHashMap<String, SagaState> is safe for concurrent access but is in-memory only. Restart = all in-progress sagas lost. See §8 Homework for DB-backed persistence.*

OrderSagaOrchestrator — Skeleton + startSaga()

OrderSagaOrchestrator.java — order-service

```
// — ENTRY POINT: start the Saga —————
public OrderResponse startSaga(OrderRequest request) {
    String orderId = UUID.randomUUID().toString();

    // Step 1: Save order in PENDING state (local transaction)
    Order order = new Order(orderId, request.getProductId(),
        request.getQuantity(), request.getAmount(), OrderStatus.PENDING);
    orderRepository.save(order);

    // Step 2: Record Saga state
    sagaStates.put(orderId, SagaState.STARTED);

    // Step 3: SEND command to Inventory Service
    // NOTE: This is a COMMAND ('please reserve'), not a domain event ('order was placed')
    kafkaTemplate.send("saga-commands", orderId,
        new ReserveInventoryCommand(orderId, request.getProductId(),
            request.getQuantity()));

    transition(orderId, SagaState.INVENTORY_RESERVING);

    return new OrderResponse(orderId, OrderStatus.PENDING, "Order received — processing...");
}
```

OrderSagaOrchestrator — Skeleton + startSaga()

OrderSagaOrchestrator.java — order-service

```
// — helper: transition state + log —————  
private void transition(String orderId, SagaState newState) {  
    SagaState old = sagaStates.put(orderId, newState);  
    log.info("[SAGA] {} {} → {}", orderId, old, newState);  
}  
}
```

startSaga() Log Trace

```
$ curl -X POST http://localhost:8080/api/orders/saga -d '{...}'

{"orderId":"ORD-789","status":"PENDING","message":"Processing..."}

[SAGA] ORD-789 null → STARTED
[SAGA] ORD-789 STARTED → INVENTORY_RESERVING

// Order saved as PENDING. Command sent. Orchestrator now waits
// for InventoryResultEvent on the saga-results topic.
```

Notice: the Orchestrator does not block. It records STARTED, sends the command, transitions to INVENTORY_RESERVING, and returns immediately.

handleInventoryResult() — Happy Path + Failure

```
// — LISTEN: Inventory reserved → send Payment command —————
@KafkaListener(topics = "saga-results", groupId = "orchestrator-inventory")
public void handleInventoryResult(InventoryResultEvent event) {
    SagaState current = sagaStates.get(event.orderId());
    if (current != SagaState.INVENTORY_RESERVING) {
        log.warn("[SAGA] Unexpected state {} for order {}", current, event.orderId());
        return; // idempotency guard
    }

    if (event.success()) {
        // Happy path: inventory OK → send payment command
        transition(event.orderId(), SagaState.INVENTORY_RESERVED);
        kafkaTemplate.send("saga-commands", event.orderId(),
            new ProcessPaymentCommand(event.orderId(), getOrderAmount(event.orderId())));
        transition(event.orderId(), SagaState.PAYMENT_PROCESSING);
    } else {
        // Inventory failed → no compensation needed, just cancel
        transition(event.orderId(), SagaState.INVENTORY_RESERVE_FAILED);
        updateOrderStatus(event.orderId(), OrderStatus.CANCELLED);
        sagaStates.remove(event.orderId()); // saga complete
    }
}
```

handlePaymentResult() — Complete or Compensate

```
// — LISTEN: Payment result → complete or compensate —————
@KafkaListener(topics = "saga-results", groupId = "orchestrator-payment")
public void handlePaymentResult(PaymentResultEvent event) {
    SagaState current = sagaStates.get(event.orderId());
    if (current != SagaState.PAYMENT_PROCESSING) return; // idempotency guard

    if (event.success()) {
        // Happy path complete
        transition(event.orderId(), SagaState.COMPLETED);
        updateOrderStatus(event.orderId(), OrderStatus.CONFIRMED);
        sagaStates.remove(event.orderId()); // saga complete
        log.info("[SAGA] □ Order {} CONFIRMED", event.orderId());
    } else {
        // Payment failed → Orchestrator decides: compensate inventory
        transition(event.orderId(), SagaState.PAYMENT_FAILED);
        kafkaTemplate.send("saga-commands", event.orderId(),
            new ReleaseInventoryCommand(event.orderId()));
        transition(event.orderId(), SagaState.INVENTORY_RELEASING);
    }
}
```

handleInventoryReleased() — Saga Cancelled

```
// — LISTEN: Inventory released → saga cancelled —————
@KafkaListener(topics = "saga-results", groupId = "orchestrator-compensation")
public void handleInventoryReleased(InventoryReleasedEvent event) {
    transition(event.orderId(), SagaState.CANCELLED);
    updateOrderStatus(event.orderId(), OrderStatus.CANCELLED);
    sagaStates.remove(event.orderId());
    log.info("[SAGA] ☐ Order {} CANCELLED — compensation complete", event.orderId());
}
```

Full Saga Log Traces — Happy Path & Compensation

Happy Path

```
STARTED
→ INVENTORY_RESERVING
→ INVENTORY_RESERVED
→ PAYMENT_PROCESSING
→ COMPLETED □
```

Order ORD-789 CONFIRMED

Compensation Path

```
STARTED
→ INVENTORY_RESERVING
→ INVENTORY_RESERVED
→ PAYMENT_PROCESSING
→ PAYMENT_FAILED
→ INVENTORY_RELEASING
→ CANCELLED □
```

Order ORD-790 CANCELLED — compensation complete

Same Scenario, Two Approaches

```
// S7 - Choreography: Inventory Service
// Inventory Service decides compensation:

@KafkaListener(topics="payment-events")
public void handlePaymentFailed(String raw) {
    if (raw.contains("PaymentFailed")) {
        // Inventory Service DECIDES
        // to release its own stock
        inventoryService.releaseStock(
            event.orderId());
        kafkaTemplate.send(
            "inventory-events",
            new InventoryReleasedEvent(
                event.orderId()));
    }
}
// Compensation logic owned by
// Inventory Service
```

```
// S12 - Orchestrator decides:
// Orchestrator decides compensation:

@KafkaListener(topics="saga-results")
public void handlePaymentResult(
    PaymentResultEvent event) {
    if (!event.success()) {
        // Orchestrator DECIDES to
        // release inventory
        kafkaTemplate.send(
            "saga-commands",
            new ReleaseInventoryCommand(
                event.orderId()));
        transition(event.orderId(),
            INVENTORY_RELEASING);
    }
}
// Compensation logic owned by
// Orchestrator - single source
```

Neither Pattern Deletes the Other — Both Coexist

Question	S7 Choreography	S12 Orchestration
Who decides to release inventory?	Inventory Service (its own listener)	Orchestrator (sends ReleaseInventoryCommand)
Where is compensation logic?	Distributed across service listeners	Single class: OrderSagaOrchestrator
Do they delete each other's code?	NO — both coexist in the repo	NO — startSaga() vs createOrder()
Which to use in production?	≤3 services, simple flow	4+ services, conditional logic, audit needed

🔗 *createOrder() (S7 Kafka events) and startSaga() (S12 Orchestrator) both exist in order-service — a real team picks one per flow.*

Lab 10A — Build OrderSagaOrchestrator

20 min in-session · complete as homework · Builds on S7 Choreography (unchanged)

1

Define Events & Commands + SagaState enum

ReserveInventoryCommand, ProcessPaymentCommand, ReleaseInventoryCommand + result events. All 9 SagaState values.

2

Build OrderSagaOrchestrator

startSaga(), handleInventoryResult(), handlePaymentResult(), handleInventoryReleased(), transition() helper.

3

Unit Tests: Verify State Transitions

Mock KafkaTemplate. Verify startSaga() → INVENTORY_RESERVING. Verify compensation sends ReleaseInventoryCommand.

What We Added Today

? Capability Added

Explicit Distributed Saga Control

One class owns the complete decision tree for Order→Inventory→Payment.
Compensation triggered by the Orchestrator, not services.

? Services Updated

order-service — OrderSagaOrchestrator + SagaState enum
S7 Choreography code — unchanged ?

? Technical Debt — Documented (deliberate scoping decision, not a bug)

In-memory SagaState (ConcurrentHashMap) is lost on restart. Fix: Persistent Saga State → homework/bonus. Full fix pattern: Event Sourcing (Phase 3, Session 19).



Docker



Testing I



Contract



Saga Orch. ✓



CI/CD



GitOps



K8s Core



K8s Adv.



DAILY QUIZ

8 Questions · 10 Minutes

Choreography vs Orchestration · SagaState · Commands vs Events

Homework & Checkpoint Commit

🔗 Definition of Done — Session 12

- ✓ SagaState enum: all 9 states present
- ✓ startSaga() transitions INVENTORY_RESERVING
- ✓ handlePaymentResult(fail) sends ReleaseInventoryCommand
- ✓ S7 Choreography tests still pass — no regressions

Commit: `session-12: add-orchestration-saga-order-service`

🔗 Pre-Session 13 Knowledge Check

Q1: Difference between a GitHub Actions job and a step?

Q2: What does 'on: push: branches: [main]' mean?

Q3: What is a Docker build-and-push step for?

🔗 **Advanced Reference (not taught):** Axon Framework provides ready-made Saga infrastructure (persistent state, event routing) but adds framework coupling. This course teaches the Orchestration PATTERN, not Axon's API — evaluate Axon independently once you understand what we built today.

5 Issues Trainees Hit — And the Fix

Issue	Solution
@KafkaListener not called — events ignored	Check groupId uniqueness: orchestrator-inventory, orchestrator-payment, orchestrator-compensation must each be distinct
SagaState transitions out of order	Add idempotency guard: if (current != EXPECTED_STATE) return; in every listener
Unit test: sagaStates map not accessible	Add package-private getSagaState(orderId) test helper, or verify via KafkaTemplate ArgumentCaptor
S7 Choreography tests fail after adding Orchestrator	Mock KafkaTemplate in tests — new @KafkaListener beans must not start real Kafka consumers
KafkaTemplate.send() fails — 'No brokers available'	Use @Mock KafkaTemplate with MockitoExtension, or @EmbeddedKafka for @SpringBootTest



SESSION 12 COMPLETE

Explicit Distributed Saga Control

Next: Session 13 — CI/CD Pipelines: GitHub Actions + Notification Service

Wednesday, 3:00–5:30 PM · Online